Namo Feature Comparison

Date: 20260521

Feature-by-feature comparison of Namo against Pandas, Polars, R/dplyr, xarray, and Julia/DataFrames.jl. Covers both where the tools are the same and where they differ.

Namo feature status is marked as **shipped**, **planned** (with version), or **not planned**.

Ingestion and schema

Schema-free instantiation

Namo — shipped (0.0.0)

Infers dimensions from hash keys. No column types, no index declaration, no schema.

```
namo = Namo.new([{symbol: 'BHP', close: 42.5}, {symbol: 'RIO', close: 118.3}])
```

Pandas — similar. Accepts a list of dicts and infers columns. Types are inferred but often wrong (strings as objects, dates as strings). However, Pandas then expects you to set an index (`df.set_index('symbol')`) to get performant lookups. Without an index, filtering is a linear scan with verbose syntax.

```
df = pd.DataFrame([{'symbol': 'BHP', 'close': 42.5}, {'symbol': 'RIO', 'close': 118.3}])
```

Polars — similar. Accepts a list of dicts. Types are inferred more aggressively than Pandas (strict typing). Schema can be passed explicitly. No index concept — Polars never requires you to declare one. Closest to Namo on ingestion simplicity.

```
df = pl.DataFrame([{'symbol': 'BHP', 'close': 42.5}, {'symbol': 'RIO', 'close': 118.3}])
```

R/dplyr — similar. Column types are inferred. No index concept in dplyr — grouping is always at the call site via `group_by()`.

```
df <- tibble(symbol = c('BHP', 'RIO'), close = c(42.5, 118.3))
```

xarray — different. Requires explicit declaration of dimensions, coordinates, and data variables at construction. You must decide what is a dimension, what is a coordinate, and what is a data variable before loading. This is the opposite of Namo's approach.

```
ds = xr.Dataset(
    {'close': (['obs'], [42.5, 118.3])},
    coords={'symbol': (['obs'], ['BHP', 'RIO'])}
)
```

Julia/DataFrames.jl — similar. Types inferred. No index concept. Clean ingestion.

```
df = DataFrame(symbol = ["BHP", "RIO"], close = [42.5, 118.3])
```

Summary: Most tools handle basic ingestion from records without much ceremony. xarray is the outlier, requiring structural decisions at ingestion. The real difference emerges in what happens next — whether the tool requires you to designate some columns as special (index, dimension, grouping key) before you can work with them.

No index/data distinction

Namo — shipped (0.0.0)

Every dimension is equal. `namo[close: 42.5]` and `namo[symbol: 'BHP']` use the same mechanism. No column is privileged as an index or demoted to "just data."

```
namo[symbol: 'BHP']
namo[close: 42.5]
```

Pandas — different. The index/column distinction is fundamental to Pandas' design. Different syntax, different performance characteristics, different behaviour on joins. `reset_index()` and `set_index()` are among the most frequently called Pandas methods, indicating that the distinction creates ongoing friction. Multi-level indexes (MultiIndex) add further complexity.

```
df.loc['BHP']                  # uses the index
df[df['close'] == 42.5]        # uses a column — different syntax
```

Polars — same as Namo. No index. All columns are equal. Filtering always uses the same expression syntax regardless of which column you filter on.

```
df.filter(pl.col('symbol') == 'BHP')
df.filter(pl.col('close') == 42.5)
```

R/dplyr — mostly same. Tibbles have row names but they're vestigial — dplyr ignores them. All columns are filtered identically via `filter()`.

```
filter(df, symbol == 'BHP')
filter(df, close == 42.5)
```

xarray — different, and more so than Pandas. Three-tier hierarchy: dimensions (axes), coordinates (labels on axes), and data variables (the payload). Each has different access patterns. You can select on coordinates but not on data variables with the same syntax.

```
ds.sel(symbol='BHP')           # works on coordinates
# ds.sel(close=42.5)           # does not work on data variables
ds.where(ds['close'] == 42.5)  # different syntax required
```

Julia/DataFrames.jl — same as Namo. No index. All columns filtered identically.

```
filter(:symbol => ==("BHP"), df)
filter(:close => ==(42.5), df)
```

Summary: Pandas and xarray impose structural hierarchies on columns. Polars, R/dplyr, and Julia/DataFrames.jl treat all columns equally, like Namo. However, none of them extend this equality to computed values (formulae) the way Namo does — see "Formulae" below.

Selection

Keyword selection

Namo — shipped (0.0.0)

The dimension name is the keyword, the criterion is the value.

```
namo[symbol: 'BHP']
namo[symbol: 'BHP', quarter: 'Q1']
```

Pandas — different. The first form requires repeating the DataFrame name and wrapping in `df[...]`. The second uses `&` instead of `and`, parentheses required due to operator precedence. The `.query()` alternative uses a string-based DSL.

```
df[df['symbol'] == 'BHP']
df[(df['symbol'] == 'BHP') & (df['quarter'] == 'Q1')]
df.query("symbol == 'BHP' and quarter == 'Q1'")
```

Polars — different. Every column reference requires `pl.col()`.

```
df.filter(pl.col('symbol') == 'BHP')
df.filter((pl.col('symbol') == 'BHP') & (pl.col('quarter') == 'Q1'))
```

R/dplyr — similar in spirit. Clean, but it's a function call, not an operator on the object. Bare column names inside `filter()` via non-standard evaluation.

```
filter(df, symbol == 'BHP')
filter(df, symbol == 'BHP', quarter == 'Q1')
```

xarray — similar syntax for coordinate selection. Keyword arguments, dimension name as keyword. But only works on coordinates, not data variables.

```
ds.sel(symbol='BHP')
```

Julia/DataFrames.jl — different. The pair syntax is unusual. Multi-column filtering requires compound lambdas.

```
filter(:symbol => ==("BHP"), df)
filter([:symbol, :quarter] => (s, q) -> s == "BHP" && q == "Q1", df)
```

Summary: Namo's keyword selection is closest to xarray's `sel()` in syntax but applies to all dimensions, not just coordinates. R/dplyr is clean but function-based. Pandas and Polars require more ceremony. Julia's pair syntax is idiosyncratic.

Array and range selection

Namo — shipped (0.0.0)

```
namo[quarter: ['Q1', 'Q2']]
namo[close: 10.0..20.0]
```

Pandas — different methods for each.

```
df[df['quarter'].isin(['Q1', 'Q2'])]
df[df['close'].between(10.0, 20.0)]
```

Polars — different methods for each.

```
df.filter(pl.col('quarter').is_in(['Q1', 'Q2']))
df.filter(pl.col('close').is_between(10.0, 20.0))
```

R/dplyr — different functions for each.

```
filter(df, quarter %in% c('Q1', 'Q2'))
filter(df, between(close, 10.0, 20.0))
```

xarray — array selection on coordinates. Range selection via `slice` but only on coordinates, not data variables.

```
ds.sel(quarter=['Q1', 'Q2'])
ds.sel(close=slice(10.0, 20.0))  # only on coordinates
```

Julia/DataFrames.jl — lambda syntax for both.

```
filter(:quarter => q -> q in ["Q1", "Q2"], df)
filter(:close => c -> 10.0 <= c <= 20.0, df)
```

Summary: All tools support array and range selection. Namo's advantage is that both are part of the same `[]` interface — arrays and ranges are just values you pass, not different method calls. The same keyword slot accepts a scalar, an array, a range, a proc (0.8.0), or a regex (0.8.0).

Proc-based selection

Namo — planned (0.8.0)

An arbitrary predicate as a selection value. Any logic expressible in Ruby.

```
namo[pe: ->(v){ v && v < 15 }]
```

Pandas — `.apply()` with a lambda is the equivalent, but it's applied to the column, not as a selection criterion. No way to pass a predicate directly to the filtering interface.

```
df[df['pe'].apply(lambda v: v is not None and v < 15)]
```

Polars — doesn't support arbitrary lambdas in filter expressions. The expression DSL is powerful but constrained to what Polars implements. Complex predicates require `map_elements()` which breaks lazy evaluation.

```
df.filter(pl.col('pe').is_not_null() & (pl.col('pe') < 15))
# Arbitrary predicates require:
df.filter(pl.col('pe').map_elements(lambda v: v is not None and v < 15))
```

R/dplyr — clean for simple predicates. Arbitrary functions work inside `filter()` via non-standard evaluation.

```
filter(df, !is.na(pe), pe < 15)
filter(df, my_predicate(pe))
```

xarray — boolean masking only, no arbitrary predicates on coordinates.

```
ds.where(ds['pe'] < 15)
```

Julia/DataFrames.jl — lambda syntax in filter, similar capability to Namo.

```
filter(:pe => v -> !ismissing(v) && v < 15, df)
```

Summary: Namo and Julia both accept arbitrary predicates inline in the selection interface. R/dplyr does too via non-standard evaluation. Pandas requires `.apply()` as a separate step. Polars restricts you to its expression DSL. xarray supports only boolean masking.

Regex-based selection

Namo — planned (0.8.0)

```
namo[symbol: /^BH/]
```

Pandas — the `.str` accessor is required to access string methods on a column.

```
df[df['symbol'].str.match(r'^BH')]
```

Polars — expression chain with string namespace.

```
df.filter(pl.col('symbol').str.contains(r'^BH'))
```

R/dplyr — function call wrapping the column reference.

```
filter(df, grepl("^BH", symbol))
```

xarray — no string pattern matching on coordinates.

Julia/DataFrames.jl — lambda with `occursin`.

```
filter(:symbol => s -> occursin(r"^BH", s), df)
```

Summary: All general-purpose tools can do regex filtering, but none accept a regex as a direct selection value. Namo treats regex as a first-class selection type alongside scalars, arrays, ranges, and procs — the same slot accepts all of them.

Mixed selection types in one call

Namo — planned (0.8.0)

Regex, range, proc, and exact value in a single `[]` call. Each dimension uses the selection type that fits.

```
namo[symbol: /^BH/, close: 10.0..50.0, pe: ->(v){ v && v < 15 }, quarter: 'Q1']
```

Pandas — four different syntactic patterns combined with `&`.

```
df[
    (df['symbol'].str.match(r'^BH')) &
    (df['close'].between(10.0, 50.0)) &
    (df['pe'].apply(lambda v: v is not None and v < 15)) &
    (df['quarter'] == 'Q1')
]
```

Polars — four different expression patterns combined with `&`.

```
df.filter(
    (pl.col('symbol').str.contains(r'^BH')) &
    (pl.col('close').is_between(10.0, 50.0)) &
    (pl.col('pe').map_elements(lambda v: v is not None and v < 15)) &
    (pl.col('quarter') == 'Q1')
)
```

R/dplyr — multiple comma-separated predicates, each using a different function. Cleaner than Pandas but still heterogeneous.

```
filter(df,
    grepl("^BH", symbol),
    between(close, 10.0, 50.0),
    !is.na(pe), pe < 15,
    quarter == 'Q1'
)
```

xarray — cannot combine selection types in a single call.

Julia/DataFrames.jl — requires a compound lambda or multiple chained filters.

```
filter(
    [:symbol, :close, :pe, :quarter] =>
        (s, c, p, q) -> occursin(r"^BH", s) && 10.0 <= c <= 50.0 &&
            !ismissing(p) && p < 15 && q == "Q1",
    df
)
```

Summary: No other tool unifies selection types into a single polymorphic interface. Namo's `[]` dispatches on the type of each value — the user doesn't need to know which method to call for arrays vs ranges vs predicates. This is a direct consequence of the `case` statement in `Row#match?`.

Projection and contraction

Projection

Namo — shipped (0.1.0)

```
namo[:symbol, :close]
```

Pandas — string column names in a list.

```
df[['symbol', 'close']]
```

Polars — method call or bracket syntax.

```
df.select(['symbol', 'close'])
```

R/dplyr — bare names via non-standard evaluation.

```
select(df, symbol, close)
```

xarray — for data variables. No direct equivalent for coordinates.

```
ds[['close']]
```

Julia/DataFrames.jl — symbol column names.

```
select(df, :symbol, :close)
```

Summary: All tools support column selection. Namo and Julia use symbols, R/dplyr uses bare names, Pandas and Polars use strings. Functionally equivalent.

Contraction

Namo — shipped (0.3.0)

Remove named dimensions. The complement of projection — say what to drop, not what to keep.

```
namo[-:symbol, -:close]
```

Pandas — method call with a keyword argument.

```
df.drop(columns=['symbol', 'close'])
```

Polars — method call.

```
df.drop(['symbol', 'close'])
```

R/dplyr — R's `select()` supports negation with `-`, directly analogous to Namo's `-:`. This is the closest parallel in any tool.

```
select(df, -symbol, -close)
```

xarray — method call.

```
ds.drop_vars(['symbol', 'close'])
```

Julia/DataFrames.jl — wrapper function.

```
select(df, Not([:symbol, :close]))
```

Summary: R/dplyr's `-column` syntax is the closest to Namo's `-:dimension`. Both express contraction as negation of names. The others use explicit drop/Not methods. Namo raises `ArgumentError` when mixing projection and contraction in the same call, enforcing a clean choice.

Combined selection and projection

Namo — shipped (0.1.0)

Positional symbols for projection and keyword arguments for selection in a single `[]` call.

```
namo[:symbol, :close, quarter: 'Q1']
```

Pandas — two operations.

```
df.loc[df['quarter'] == 'Q1', ['symbol', 'close']]
```

Polars — two operations chained.

```
df.filter(pl.col('quarter') == 'Q1').select(['symbol', 'close'])
```

R/dplyr — two operations chained via pipe.

```
df %>% filter(quarter == 'Q1') %>% select(symbol, close)
```

xarray — two operations. Selection on coordinates, then variable selection.

```
ds.sel(quarter='Q1')[['close']]
```

Julia/DataFrames.jl — two operations or a compound expression.

```
select(filter(:quarter => ==("Q1"), df), :symbol, :close)
```

Summary: No other tool combines selection and projection in a single call. All require two operations, either chained or nested. Namo's `[]` overloading handles both in one expression.

Formulae

Computed dimensions

Namo — shipped (0.1.0)

A formula is a named computation that resolves lazily per row. Once defined, it's indistinguishable from a data dimension — `row[:revenue]` works the same whether `revenue` is stored data or a formula.

```
namo[:revenue] = proc{|row| row[:price] * row[:quantity]}
```

Pandas — computes immediately and stores the result as a new column. The result is data, not a formula — it doesn't update if the underlying columns change.

```
df['revenue'] = df['price'] * df['quantity']
```

Polars — expression-based, lazy in lazy mode, but the result is still a materialised column. No concept of a formula that re-evaluates per row.

```
df = df.with_columns((pl.col('price') * pl.col('quantity')).alias('revenue'))
```

R/dplyr — computes immediately. The `revenue` column is data from this point forward.

```
df <- df %>% mutate(revenue = price * quantity)
```

xarray — immediate computation on arrays.

```
ds['revenue'] = ds['price'] * ds['quantity']
```

Julia/DataFrames.jl — immediate computation with broadcasting.

```
df.revenue = df.price .* df.quantity
```

Summary: Every other tool computes immediately and stores the result. Namo's formulae are lazy — they resolve when accessed, not when defined. This means formulae can reference other formulae that haven't been defined yet (forward references), and they always reflect the current state of dependencies. No other tool in this comparison has this capability.

Formula chains

Namo — shipped (0.1.0)

Each formula references the previous. The chain resolves lazily through Row.

```
namo[:revenue] = proc{|row| row[:price] * row[:quantity]}
namo[:profit] = proc{|row| row[:revenue] - row[:cost]}
namo[:margin] = proc{|row| row[:profit] / row[:revenue]}
```

Pandas — each must be computed in order. If you define them out of order, the column doesn't exist yet and you get a `KeyError`. The dependency order is the user's responsibility.

```
df['revenue'] = df['price'] * df['quantity']
df['profit'] = df['revenue'] - df['cost']
df['margin'] = df['profit'] / df['revenue']
```

Polars — `with_columns` calls must be sequenced correctly, or combined in a single `with_columns` that Polars evaluates in declaration order.

```
df = df.with_columns((pl.col('price') * pl.col('quantity')).alias('revenue'))
df = df.with_columns((pl.col('revenue') - pl.col('cost')).alias('profit'))
df = df.with_columns((pl.col('profit') / pl.col('revenue')).alias('margin'))
```

R/dplyr — `mutate()` evaluates sequentially within a single call. This works because `mutate` processes columns in order. But it's still eager evaluation.

```
df <- df %>% mutate(
    revenue = price * quantity,
    profit = revenue - cost,
    margin = profit / revenue
)
```

xarray — sequential assignment, order matters.

```
ds['revenue'] = ds['price'] * ds['quantity']
ds['profit'] = ds['revenue'] - ds['cost']
ds['margin'] = ds['profit'] / ds['revenue']
```

Julia/DataFrames.jl — sequential, order matters.

```
df.revenue = df.price .* df.quantity
df.profit = df.revenue .- df.cost
df.margin = df.profit ./ df.revenue
```

Summary: R/dplyr's `mutate()` handles sequential dependencies within a single call. Namo handles them lazily across any number of separate definitions, in any order. No other tool allows forward references — defining `profit` before `revenue` and having it resolve correctly when accessed.

Formulae indistinguishable from data

Namo — shipped (0.1.0)

`row[:close]` and `row[:earnings_yield]` resolve through the same mechanism. The consumer doesn't know or care whether a value is stored or computed.

Pandas — computed columns become data columns. They look the same in the DataFrame, but they were computed eagerly. If underlying data changes, computed columns are stale. The user must re-run the computation. No way to mark a column as "derived" versus "stored."

Polars — same. Once materialised, a computed column is indistinguishable from data, but it's a snapshot, not a live computation.

R/dplyr — same.

xarray — same.

Julia/DataFrames.jl — same.

Summary: All tools make computed values look like data after materialisation. Only Namo keeps them as live computations that re-evaluate on access. This is the fundamental architectural difference.

Two-arity formulae

Namo — planned (0.11.0)

A formula that references other rows in the same dataset.

```
namo[:sma_20] = proc do |row, namo|
  window = namo[symbol: row[:symbol], date: ..row[:date]].last(20)
  window.sum{|r| r[:close]} / window.length.to_f
end
```

Pandas — the `groupby` + `transform` + `rolling` chain is the idiom. Powerful but the formula is not attached to the dataset — it's a one-off column creation.

```
df['sma_20'] = df.groupby('symbol')['close'].transform(lambda x: x.rolling(20).mean())
```

Polars — expression DSL with window functions. Clean but constrained to what Polars implements.

```
df = df.with_columns(
    pl.col('close').rolling_mean(20).over('symbol').alias('sma_20')
)
```

R/dplyr — requires the `zoo` package for rolling operations.

```
df <- df %>% group_by(symbol) %>% mutate(sma_20 = zoo::rollmean(close, 20, fill = NA))
```

xarray — rolling operations are built in for numeric data along named dimensions.

```
ds['sma_20'] = ds['close'].rolling(date=20).mean()
```

Julia/DataFrames.jl — requires a rolling mean function.

```
combine(groupby(df, :symbol), :close => (x -> rollmean(x, 20)) => :sma_20)
```

Summary: All tools can compute rolling windows, but through different mechanisms — groupby chains, expression DSLs, or external packages. Namo's two-arity formula is more general: the proc receives the entire Namo and can perform arbitrary cross-row logic, not just windowed aggregations. The selection `namo[symbol: row[:symbol], date: ..row[:date]]` inside the formula uses the same `[]` interface as external selection.

Parameterised formulae

Namo — planned (0.12.0)

A single formula definition that works across different fields and parameters.

```
namo[:sma] = proc do |row, namo, field, period|
  window = namo[symbol: row[:symbol], date: ..row[:date]].last(period)
  window.sum{|r| r[field]} / window.length.to_f
end

row[:sma, :close, 20]
row[:sma, :volume, 50]
```

Pandas — no equivalent. You'd write a function and call it per column. The function is external to the DataFrame.

```
def sma(df, field, period):
    return df.groupby('symbol')[field].transform(lambda x: x.rolling(period).mean())

df['sma_close_20'] = sma(df, 'close', 20)
df['sma_volume_50'] = sma(df, 'volume', 50)
```

Polars — no equivalent. Expression DSL doesn't support parameterised reuse in this way.

```
df = df.with_columns(pl.col('close').rolling_mean(20).over('symbol').alias('sma_close_20'))
df = df.with_columns(pl.col('volume').rolling_mean(50).over('symbol').alias('sma_volume_50'))
```

R/dplyr — you'd write a function and use it inside `mutate()`. The function is external.

```
sma <- function(x, n) zoo::rollmean(x, n, fill = NA)

df <- df %>% group_by(symbol) %>% mutate(
    sma_close_20 = sma(close, 20),
```

```
    sma_volume_50 = sma(volume, 50)
)
```

xarray — no equivalent.

```
ds['sma_close_20'] = ds['close'].rolling(date=20).mean()
ds['sma_volume_50'] = ds['volume'].rolling(date=50).mean()
```

Julia/DataFrames.jl — same as R. External function, applied per column.

```
sma(x, n) = rollmean(x, n)

combine(groupby(df, :symbol),
    :close => (x -> sma(x, 20)) => :sma_close_20,
    :volume => (x -> sma(x, 50)) => :sma_volume_50
)
```

Summary: No other tool has parameterised formulae as a first-class concept attached to the dataset. In every other tool, reusable computations are external functions that you call during column creation. In Namo, they're named formulae on the dataset that accept arguments at access time.

Composition

Equi-join

Namo — shipped (0.9.0)

Joins on shared dimensions automatically.

```
ohlcv * fundamentals
```

Pandas — you must specify the join columns. Or Pandas infers shared column names with a warning.

```
pd.merge(ohlcv, fundamentals, on=['symbol', 'exchange'])
```

Polars — must specify join columns.

```
ohlcv.join(fundamentals, on=['symbol', 'exchange'])
```

R/dplyr — must specify join columns. Or `inner_join(ohlcv, fundamentals)` infers shared columns but warns.

```
inner_join(ohlcv, fundamentals, by = c('symbol', 'exchange'))
```

xarray — merges on shared dimensions automatically. Closest to Namo's $*$ in concept, but restricted to the dimension/coordinate/variable hierarchy.

```
xr.merge([ds1, ds2])
```

Julia/DataFrames.jl — must specify join columns.

```
innerjoin(ohlcv, fundamentals, on = [:symbol, :exchange])
```

Summary: Most tools require explicit join column specification. R/dplyr can infer shared columns but warns. xarray merges on shared dimensions automatically. Namo's $*$ infers shared dimensions silently — the operator is the join, and the shared dimension names are the join condition.

Conditional join with block

Namo — planned (0.10.0)

Custom matching logic — find the most recent quarterly report as of each daily observation.

```
ohlcv.*(fundamentals) do |row, candidates|
  candidates.select{|f| f[:quarter_end] <= row[:date]}.max_by{|f| f[:quarter_end]}
end
```

Pandas — a specialised function that most users don't know exists. Requires both DataFrames to be sorted by the join key. The `direction` parameter is limited to 'backward', 'forward', and 'nearest'.

```
pd.merge_asof(ohlcv, fundamentals,
    left_on='date', right_on='quarter_end',
    by='symbol', direction='backward'
)
```

Polars — similar to Pandas' `merge_asof`.

```
ohlcv.join_asof(fundamentals,
    left_on='date', right_on='quarter_end',
    by='symbol', strategy='backward'
)
```

R/dplyr — no built-in asof join. Workarounds involve `fuzzyjoin` or rolling joins via `data.table`. The tidyverse has no native solution.

xarray — no equivalent. Merging assumes aligned coordinates.

Julia/DataFrames.jl — no built-in asof join. Workarounds via `LeftJoin` + filtering or external packages.

Summary: Pandas and Polars have specialised asof join functions with fixed strategies (backward, forward, nearest). Namo's block on $*$ is fully general — the matching logic is arbitrary Ruby code. Any temporal, spatial, or domain-specific matching strategy is expressible. The asof join is just one use case.

Cartesian product

Namo — shipped (0.9.0)

```
ohlcv ** fundamentals
```

Pandas — added in Pandas 1.2. Before that, required adding a dummy key column.

```
pd.merge(ohlcv, fundamentals, how='cross')
```

Polars

```
ohlcv.join(fundamentals, how='cross')
```

R/dplyr — `crossing()` from tidyr.

```
crossing(ohlcv, fundamentals)
```

xarray — no direct Cartesian product of datasets.

Julia/DataFrames.jl

```
crossjoin(ohlcv, fundamentals)
```

Summary: All general-purpose tools support cross joins, though the syntax varies. Namo's $**$ is the most concise. The visual relationship between $*$ (filtered) and $**$ (unfiltered) communicates the distinction — more sigil, more output.

Decomposition

Namo — shipped (0.9.0)

Factor out dimensions. The inverse of $*$ and $**$.

```
combined / ohlcv
```

Pandas — no equivalent operation. You'd manually drop columns and deduplicate.

```
# No direct equivalent
combined.drop(columns=[c for c in ohlcv.columns if c not in shared]).drop_duplicates()
```

Polars — no equivalent.

R/dplyr — no equivalent. You'd `select()` the desired columns and `distinct()`.

```
# No direct equivalent
combined %>% select(-exclusive_to_ohlcv_cols) %>% distinct()
```

xarray — no equivalent.

Julia/DataFrames.jl — no equivalent.

Summary: No other tool exposes decomposition as a first-class operator — the concept of "undoing" a join, factoring out the dimensions contributed by one operand, doesn't exist elsewhere. This completes the algebraic relationship: `(a ** b) / b` recovers `a` exactly, and `(a * b) / b` recovers `a` modulo the dimensions shared with `b`.

Where Namo's decomposition operator differs structurally from `*` and `**` is in its precondition stance. `*` and `**` are strict — they raise on dimension-incompatible operands, because combining unrelated Namos has no natural answer and silently producing arbitrary output would turn a logic error into nonsense rows. `/` is loose — it's a no-op when the operands share no dimensions, because projecting away nothing returns the original. This asymmetry isn't arbitrary; it reflects a structural distinction between combining and projecting. The asymmetry earns `/` properties a strict version would lose: identity test (`c / b == c` iff dimensionally independent), idempotence (`(c / b) / b == c / b`), and pipeline composition (a `/ separator` step runs over any Namo without special-casing applicability). The pattern mirrors `Array#-` — `[1, 2, 3] - [9] == [1, 2, 3]`, not an error — where the no-op-on-non-applicable rule lets the operator compose into pipelines that don't know in advance whether the operation applies.

Set operators

Row-level set algebra

Namo — shipped (0.4.0–0.5.0)

All five operate on whole rows, require matching dimensions, and carry formulae through.

```
a + b    # concatenation
a - b    # row removal
a & b    # intersection
a | b    # union (deduplicated)
a ^ b    # symmetric difference
```

Pandas — concatenation via function call. Set operations require workarounds via merges and deduplication. No operator syntax. No symmetric difference.

```
pd.concat([a, b])                                    # concatenation
pd.merge(a, b, how='inner')                          # intersection (approximate)
pd.concat([a, b]).drop_duplicates()                  # union (approximate)
# difference and symmetric difference require merge + indicator + filter
```

Polars — concatenation via function call. No set operators. Workarounds via joins and anti-joins.

```
pl.concat([a, b])
# Set operations require join workarounds
```

R/dplyr — named functions for three of five. No symmetric difference. No operator syntax.

```
bind_rows(a, b)      # concatenation
intersect(a, b)      # intersection
union(a, b)          # union
setdiff(a, b)        # difference
# No symmetric difference
```

xarray — concatenation along a dimension. No row-level set operations.

```
xr.concat([ds1, ds2], dim='obs')
```

Julia/DataFrames.jl — concatenation via function call. Set operations via converting to sets of rows. No symmetric difference. No operator syntax.

```
vcat(a, b)
# intersect, union, setdiff via Sets
```

Summary: R/dplyr comes closest with `intersect`, `union`, `setdiff` as named functions. Namo provides all five as operators that read like algebraic expressions and compose naturally. No other tool has symmetric difference as a first-class operation. No other tool carries formulae through set operations.

Blocks on set operators

Namo — planned (0.10.0)

Relax exact row matching to a custom predicate.

```
today.-(exclusions){|a, b| a[:symbol] == b[:symbol]}
```

Pandas — anti-join on a column. Works for single-column matching. Multi-column or complex matching requires merge with indicator.

```
today[~today['symbol'].isin(exclusions['symbol'])]
```

Polars — anti-join syntax.

```
today.join(exclusions, on='symbol', how='anti')
```

R/dplyr — clean, but the `by` parameter only accepts column names, not arbitrary predicates.

```
anti_join(today, exclusions, by = 'symbol')
```

xarray — no equivalent.

Julia/DataFrames.jl — same as R.

```
antijoin(today, exclusions, on = :symbol)
```

Summary: Other tools handle column-name-based matching via anti-joins. Namo's blocks accept arbitrary predicates — the matching logic is not restricted to column equality. A block could match on computed values, fuzzy comparisons, or any other criterion.

Worked example: comparing yesterday's screen to today's

A daily screen produces a set of candidates. Comparing today's result to yesterday's — unchanged, grown, shrunk, or churned — reads as a sequence of set-operator calls.

Namo — shipped (0.4.0–0.6.0)

```
status = if today == yesterday; "no change"
      elsif today >= yesterday; "added #{(today - yesterday).count} candidates"
      elsif yesterday >= today; "removed #{(yesterday - today).count} candidates"
      else;                     "churn: #{(today ^ yesterday).count} differing"
end
```

The row-set operators (==, <=, >=, –, &, |, ^) all work directly on Namos and return Namos (or booleans). The whole comparison stays in one type system. The algebraic identity `today >= yesterday` iff `(yesterday - today).count == 0` reads consistently with how you'd describe a superset relation in English.

Pandas — `DataFrame.equals` is order-sensitive, so set-equality on rows requires materialising each row as a tuple and dropping into Python's `set` type. The DataFrame structure goes away for the operation; the boolean comes back as a Python primitive. The `set` type happens to have ==, >=, –, ^ defined, which is why this works at all — but you're working in two type systems for one comparison.

```
yesterday_rows = set(map(tuple, yesterday.itertuples(index=False)))
today_rows     = set(map(tuple, today.itertuples(index=False)))

if   today_rows == yesterday_rows: status = "no change"
elif today_rows >= yesterday_rows: status = f"added {len(today_rows - yesterday_rows)} candidates"
elif yesterday_rows >= today_rows: status = f"removed {len(yesterday_rows - today_rows)} candidates"
else:                         status = f"churn: {len(today_rows ^ yesterday_rows)} differing"
```

Polars / **R/dplyr** / **xarray** / **Julia** — same pattern as Pandas. None has full set algebra on whole datasets; the workaround is to extract rows into a language-level set type, perform the operation there, and translate back.

Summary: The diff idiom is one operator call per branch in Namo; in every other tool it's a conversion to a set-of-rows type plus the operation. Having the set algebra as first-class Namo operators is what keeps the idiom short and in one type system.

Type handling

Type agnosticism

Namo — shipped (0.1.0)

Formulae work on any Ruby type. String concatenation, date arithmetic, boolean logic, pattern matching, and custom objects are all valid formula outputs. Selection works identically on all types.

```
namo[:label] = proc{|row| "#{row[:symbol]} (#{row[:exchange]})"}
namo[:status] = proc{|row| row[:volume] > 0 ? 'active' : 'suspended'}
namo[status: 'active']
```

Pandas — DataFrames hold mixed types (object columns). But the computation model assumes numeric columns. Each type has its own access pattern.

```
df['label'] = df['symbol'] + ' (' + df['exchange'] + ')'
df['status'] = df['volume'].apply(lambda v: 'active' if v > 0 else 'suspended')
df[df['status'] == 'active']
# String operations require .str accessor
# Date operations require .dt accessor
```

Polars — strict typing. Each column has a declared type. String, numeric, date, boolean columns have different expression APIs.

```
df = df.with_columns(
    (pl.col('symbol') + ' (' + pl.col('exchange') + ')').alias('label')
)
# pl.col('name').str.contains()  vs  pl.col('price') > 10
```

R/dplyr — fully type-agnostic within `mutate()`. Closest to Namo on type flexibility.

```
df <- df %>% mutate(
    label = paste(symbol, "(", exchange, ")"),
    status = ifelse(volume > 0, 'active', 'suspended')
```

```
)
filter(df, status == 'active')
```

xarray — numeric only. Coordinates can be strings or dates, but data variables are expected to be numeric arrays. String-valued data variables are technically possible but unsupported by most operations.

Julia/DataFrames.jl — columns are typed arrays. Mixed operations work but require type-appropriate functions. Julia's multiple dispatch handles type-specific operations elegantly, but the user must be type-aware.

```
df.label = df.symbol .* " (" .* df.exchange .* ")"
df.status = ifelse.(df.volume .> 0, "active", "suspended")
filter(:status => ==("active"), df)
```

Summary: R/dplyr and Namo are the most type-agnostic. Pandas and Polars partition their APIs by type. xarray is numeric-only. Namo's advantage over R is that formulae are attached to the dataset and resolve lazily, while R's `mutate()` is eager.

Enumerable integration

Yields formula-aware objects

Namo — shipped (0.2.0)

The `each` method yields Row objects that see formulae as if they were data.

```
namo[:revenue] = proc{|row| row[:price] * row[:quantity]}
total = namo.reduce(0){|sum, row| sum + row[:revenue]}
```

Pandas — yields (index, Series) pairs. Formulae don't exist — computed columns are materialised data. Iterating is discouraged for performance reasons.

```
for idx, row in df.iterrows():
    # row['revenue'] works but was eagerly computed
```

Polars — yields tuples. No formula concept. Iteration is discouraged; the expression DSL is preferred.

```
for row in df.iter_rows(named=True):
    # row['revenue'] was eagerly computed
```

R/dplyr — no row-level iteration idiom. R operates on columns, not rows. `rowwise()` exists but is slow and discouraged.

```
df %>% rowwise() %>% mutate(total = revenue + tax)
```

xarray — no row-level iteration. Operations are array-level.

Julia/DataFrames.jl — `eachrow()` yields DataFrameRow objects. No formula concept.

```
for row in eachrow(df)
    # row.revenue was eagerly computed
end
```

Summary: Only Namo yields objects that carry live formulae. In every other tool, iteration yields raw data — computed columns are already materialised. Namo's Row objects are the formula resolution mechanism, not just data containers.

Enumerable methods return Namos

Namo — planned (0.14.0)

```
filtered = namo.select{|row| row[:close] > 40.0}
filtered[symbol: 'BHP']  # works — filtered is a Namo
```

Pandas — filtering returns a DataFrame. Subsequent operations work.

```
filtered = df[df['close'] > 40]
filtered[filtered['symbol'] == 'BHP']  # works
```

Polars — `filter()` returns a DataFrame.

```
filtered = df.filter(pl.col('close') > 40)
```

R/dplyr — `filter()` returns a tibble.

```
filtered <- filter(df, close > 40)
```

xarray — `where()` returns a Dataset.

```
filtered = ds.where(ds['close'] > 40)
```

Julia/DataFrames.jl — `filter()` returns a DataFrame.

```
filtered = filter(:close => c -> c > 40, df)
```

Summary: Every other tool already returns its own type from filtering operations. Namo 0.14.0 brings it to parity. The difference is that Namo's Enumerable integration means `select`, `reject`, `sort_by` — Ruby's standard collection methods — also return Namos, not just Namo-specific filter methods.

Comparisons

Equality hierarchy

Namo — shipped (0.6.0)

A four-level equality hierarchy mirroring Ruby's standard convention, extended with === for pattern-match dispatch. Each operator answers a distinct question:

```
a.equal?(b)   # same object (object identity, inherited)
a.eql?(b)     # same class + same data (as multisets) + same formula names
a == b        # same data (as multisets), any class, formulae ignored
a === b       # same dimensions + same formula names, any class, data ignored
```

== is multiset-theoretic on rows — two Namos with the same rows in different orders are equal, but `[{x:1}, {x:1}]` is not equal to `[{x:1}]` (duplicate rows count as data). Class is ignored, formulae are ignored.

=== is the pattern-match operator. It asks "do these two Namos have the same analytical shape?" — same dimensions, same formula names, regardless of data. This is what case statements use, so a Namo can serve as a template for `case`/`when` dispatch on schema rather than on data.

`eql?` is the strictest user-facing equality. Class must match (so `TradingAnalysis.new(data).eql?(Namo.new(data))` is false even when data matches) and formula names must match. The convention follows Ruby's numerics — `1 == 1.0` is true, `1.eql?(1.0)` is false.

`hash` is consistent with `eql?` and computed from canonical form, so two Namos that are `eql?` produce the same hash and can be used as Hash keys or Set members reliably (when frozen).

Note that proc identity is deliberately *not* part of any of these operators. Two independently-written procs with identical bodies (`proc{|r| r[:x] * 2}` typed twice) are not == in Ruby, so comparing the formulae hashes directly would treat structurally-equivalent Namos as unequal. === and `eql?` therefore compare formula *names* — proc bodies are out of scope for Ruby's reflection in any practical way.

Pandas — `DataFrame.equals(other)` exists but is order-sensitive (rows in different orders compare as unequal). == does element-wise comparison, returning a DataFrame of booleans rather than a single answer. No analogue to `eql?`.

```
a.equals(b)         # element-wise, order-sensitive
(a == b).all().all() # workaround
```

Polars — `frame_equal()` is order-sensitive. `equals()` does the same. No three-tier hierarchy.

```
a.equals(b)
```

R/dplyr — base R's `identical(a, b)` checks deep equality including order. `dplyr::all_equal()` was deprecated; the current advice is to use `waldo::compare()`. No order-insensitive frame-level equality as a primitive.

```
identical(a, b)
all.equal(a, b)
```

xarray — `Dataset.equals(other)` checks coordinate, variable, and attribute equality. Order matters for coordinates.

```
ds1.equals(ds2)
```

Julia/DataFrames.jl — == does element-wise comparison; `isequal(a, b)` is the closer analogue but is also order-sensitive.

```
isequal(a, b)
```

Summary: No other tool has a four-level equality hierarchy on datasets. Most have one method that does element-wise or strict-identical comparison; none distinguish "same data ignoring order and metadata" from "same analytical shape regardless of data" from "same in every observable way." Namo's hierarchy is the only one that lets users pick the level of strictness appropriate to the question they're asking, and === makes Namos work naturally as templates in `case` dispatch.

Schema dispatch on incoming data feeds

A worked example of ===: a function receives data and needs to dispatch on its analytical shape — same dimensions and formulae mean "the same kind of analysis," regardless of the specific rows.

Namo — shipped (0.6.0)

=== makes a Namo work as a template in `case`/`when`. The template is itself a Namo — same first-class object as everything else in the program. Adding a third shape means adding a `when` clause; if the template needs to carry formulae for downstream processing, the template already does.

```
ohlcv_shape     = Namo.new([{date: nil, symbol: nil, open: 0.0, high: 0.0, low: 0.0, close: 0.0, volume: 0
fundamentals_shape = Namo.new([{symbol: nil, pe: 0.0, book_value: 0.0, dividend_yield: 0.0}])

case incoming
when ohlcv_shape        then process_ohlcv(incoming)
when fundamentals_shape then process_fundamentals(incoming)
else raise ArgumentError, "unknown shape: #{incoming.dimensions}"
end
```

Pandas — no schema-as-value. Maintain free-floating sets of column names; dispatch via `if/elif` with set subset checks.

```
ohlcv_cols        = {'date', 'symbol', 'open', 'high', 'low', 'close', 'volume'}
fundamentals_cols = {'symbol', 'pe', 'book_value', 'dividend_yield'}

incoming_cols = set(incoming.columns)

if   ohlcv_cols.issubset(incoming_cols):        process_ohlcv(incoming)
elif fundamentals_cols.issubset(incoming_cols): process_fundamentals(incoming)
else: raise ValueError(f"unknown shape: {incoming.columns.tolist()}")
```

Polars — same pattern as Pandas. `df.columns` is a list of names; dispatch via `if/elif` on set subset checks.

R/dplyr — same pattern. `names(df)` gives column names; dispatch via `if/else` on `setdiff` or `%in%`.

xarray — split namespace: `ds.dims` for dimensions, `ds.data_vars` for variables. Dispatch via `if/else` checking each separately.

Julia/DataFrames.jl — same pattern. `names(df)` gives column names; dispatch via `if/else` on set operations.

Summary: No other tool has a schema as a first-class value. Everywhere else the schema is a list of column names extracted from the dataset and checked via set operations in control flow. Namo's `===` puts the schema-template directly into Ruby's case-statement dispatch, alongside `Integer === 5`. The template is also a Namo, so it composes with everything else — carries formulae, can be selected against, can be combined with operators. Extending the dispatch beyond simple shape-checking comes for free.

Subset/superset tests

Namo — shipped (0.6.0)

```
a < b   # strict subset
a <= b  # subset
a > b   # strict superset
a >= b  # superset
```

Multiset-theoretic on rows: duplicate rows count, so a single `{x:1}` is a proper subset of two `{x:1}`s. Pair with the set operators algebraically: `a & b == a` iff `a <= b`. Following stdlib `Set`'s precedent for mapping mathematical subset notation onto Ruby's comparison operators, generalised to multisets.

Pandas — no built-in subset/superset test on DataFrames. Workarounds via merge and length comparison.

```
# No direct equivalent
is_subset = len(pd.merge(a, b, how='inner')) == len(a)
```

Polars — no built-in. Workaround via anti-join — if the anti-join is empty, a is a subset of b.

```
# No direct equivalent
is_subset = a.join(b, on=a.columns, how='anti').height == 0
is_equal = is_subset and b.join(a, on=b.columns, how='anti').height == 0
```

R/dplyr — no built-in for data frames.

```
# No direct equivalent
is_subset <- nrow(intersect(a, b)) == nrow(a)
```

xarray — no equivalent. Datasets don't have row-level identity, so subset/superset doesn't apply directly. You'd compare coordinate values and data variable contents separately.

```
# No direct equivalent
# Would need to compare each variable independently:
all(ds1[var].equals(ds2[var]) for var in ds1.data_vars)
```

Julia/DataFrames.jl — no built-in. Converting to sets of named tuples is the workaround.

```
# No direct equivalent
a_set = Set(eachrow(a))
b_set = Set(eachrow(b))
is_subset = issubset(a_set, b_set)
```

Summary: No other tool provides algebraic comparison operators on datasets. Namo's comparisons pair with the set operators: if `a & b == a`, then `a <= b`. This algebraic consistency doesn't exist elsewhere because the set operators don't exist elsewhere as first-class operations.

Introspection

Dimensions, coordinates, values

Namo — `dimensions` and `coordinates` shipped (0.0.0), `values` planned (0.7.0)

Three introspection methods forming a complete set: `dimensions` tells you what names exist, `coordinates` tells you the unique values per dimension, `values` tells you all values for a dimension.

```
namo.dimensions             # => [:symbol, :close]
namo.coordinates[:symbol]   # => ['BHP', 'RIO']
namo.values[:symbol]        # => ['BHP', 'RIO', 'BHP']
```

Pandas — columns and unique values available. No single method for "all values of a column preserving duplicates" — that's just accessing the column.

```
df.columns.tolist()         # dimensions
df['symbol'].unique()       # coordinates
df['symbol'].tolist()       # values (just the column)
```

Polars — similar to Pandas.

```
df.columns                  # dimensions
df['symbol'].unique()       # coordinates
df['symbol'].to_list()      # values
```

R/dplyr — similar.

```
colnames(df)                # dimensions
unique(df$symbol)           # coordinates
df$symbol                   # values
```

xarray — dimensions, coordinates, and data variables are separate concepts with separate accessors.

```
ds.dims                     # dimensions
ds.coords['symbol'].values  # coordinates
ds['close'].values          # values (data variable, numeric array)
```

Julia/DataFrames.jl — similar to Pandas.

```
names(df)                   # dimensions
unique(df.symbol)           # coordinates
df.symbol                   # values
```

Summary: All tools can answer these questions. The difference is conceptual: in Namo, `coordinates` and `values` are named concepts with distinct meanings (unique axis labels vs raw column data). In other tools, you just access the column and optionally call `unique()`. xarray is the only other tool where "coordinates" is a named concept, but it's part of a three-tier hierarchy rather than a simple pair.

`to_h` and columnar output

Namo — planned (0.7.0)

`values` and `to_h` produce identical output: a hash of dimension array, with row order preserved. Either form is available; both are public.

```
namo.values   # => {symbol: ['BHP', 'RIO', 'BHP'], close: [42.5, 118.3, 43.1]}
namo.to_h     # => same shape
```

The two names exist so users can reach for whichever fits the context — `values` reads naturally as inspection (`namo.values[:close].sum`), `to_h` reads naturally as conversion (`hash = namo.to_h; hash.keys`).

Pandas — `df.to_dict()` has multiple orientations. The closest analogue is `df.to_dict('list')`.

```
df.to_dict('list')   # {'symbol': ['BHP', ...], 'close': [42.5, ...]}
```

Polars — `df.to_dict()` returns the columnar shape directly.

```
df.to_dict(as_series=False)
```

R/dplyr — `as.list(df)` returns a named list of column vectors.

```
as.list(df)
```

xarray — `ds.to_dict()` returns a nested dict with metadata. Less direct.

```
ds.to_dict()
```

Julia/DataFrames.jl — no direct equivalent; you'd build the hash manually.

```
Dict(name => df[!, name] for name in names(df))
```

Summary: Most tools have a columnar-dict conversion, with varying ergonomics. Namo's contribution is making `values` and `to_h` synonyms — the inspection method and the conversion method produce the same shape, so users don't have to decide which to learn.

Aspect classes and template-matching

Namo — planned (0.7.0)

`dimensions`, `coordinates`, and `values` return subclass instances of plain Ruby types (`Namo::Dimensions < Array`, `Namo::Coordinates < Hash`, `Namo::Values < Hash`). Each subclass overrides `===` to template-match against whole Namos, enabling case-statement dispatch on schema.

```
case incoming
when ohlcv.dimensions     then process_ohlcv(incoming)
when fundamentals.dimensions then process_fundamentals(incoming)
end
```

The template-match is a structural conformance check: `ohlcv.dimensions === incoming` asks "does `incoming` have at least these dimensions?" Following Ruby's `===` convention where the left side is a template and the right side is checked against it (`Integer === 5`, `(1..10) === 5`).

For all other purposes, the aspects behave as plain Arrays and Hashes — `a.dimensions == b.dimensions` does array equality, `a.coordinates[:close]` is plain Hash indexing, and so on. The subclass identity matters only for the asymmetric template-match against Namos.

Pandas — no equivalent. Schema-based dispatch requires writing predicate functions and an `if/elif` chain.

```
def matches_ohlcv(df):
   return all(col in df.columns for col in ['symbol', 'date', 'open', 'high', 'low', 'close', 'volume'])

if matches_ohlcv(incoming):
    process_ohlcv(incoming)
elif matches_fundamentals(incoming):
    process_fundamentals(incoming)
```

Polars — same as Pandas. No first-class schema-as-pattern construct.

```
if {'symbol', 'date', 'open', 'high', 'low', 'close', 'volume'}.issubset(incoming.columns):
    process_ohlcv(incoming)
```

R/dplyr — no equivalent. Predicate functions and `if/else if` chains.

```
matches_ohlcv <- function(df) all(c('symbol', 'date', 'open', 'high', 'low', 'close', 'volume') %in% colna

if (matches_ohlcv(incoming)) {
```

```
    process_ohlcv(incoming)
}
```

xarray — no equivalent at the dataset level. xarray Datasets carry their schema (dims, coords, data_vars) but don't support pattern-matching against another Dataset's schema as a first-class operation.

Julia/DataFrames.jl — Julia's multiple-dispatch and `Match.jl` package provide pattern matching capabilities, but not for DataFrame schemas specifically. You'd write predicate functions.

Summary: Namo's aspect classes turn schema into something you can pattern-match against using Ruby's standard case-statement machinery. No other tool exposes the schema as a pattern-matching object. The closest equivalent in any tool is writing predicate functions and `if/else` chains. This works because Namo's aspects are first-class objects that respond to `===` as template-match — a construction that fits Ruby's conventions exactly but has no analogue in Python, R, or Julia's data-frame ecosystems.

What Namo doesn't have

Features present in competitors that Namo lacks or has deferred.

Aggregation / group-by-aggregate

Pandas — `df.groupby('symbol')['close'].mean()`. Full aggregation framework.

Polars — `df.group_by('symbol').agg(pl.col('close').mean())`.

R/dplyr — `df %>% group_by(symbol) %>% summarise(mean_close = mean(close))`.

xarray — `ds.groupby('symbol').mean()`.

Julia/DataFrames.jl — `combine(groupby(df, :symbol), :close => mean)`.

Namo — at 1.x, no built-in aggregation returning a Namo. Ruby's `Enumerable#group_by` works but returns a raw hash of `{key => Array<Row>}`.

```
namo.group_by{|row| row[:symbol]}.transform_values{|rows| rows.sum{|r| r[:close]} / rows.length}
# => {'BHP' => 42.8, 'RIO' => 118.3}
```

Planned for 2.x: `group_by` returns `{key => Namo}` — each group becomes a queryable Namo retaining the parent's formulae. Combined with bare names (also 2.x), aggregation pipelines become Namo-native:

```
# 2.x
namo.group_by(&:symbol).transform_values{|n| n.close.sum / n.length}
# => {'BHP' => 42.8, 'RIO' => 118.3}
```

The result is still a hash (since `group_by` produces a hash by definition), but its values are Namos rather than raw arrays, so any per-group operation can use Namo's full vocabulary.

Pivoting / reshaping

Pandas — `pivot_table`, `melt`, `stack`, `unstack`.

Polars — `pivot`, `melt`.

R/dplyr — `pivot_wider`, `pivot_longer` (tidyr).

xarray — no pivot, but dimension manipulation via `stack`/`unstack`.

Julia/DataFrames.jl — `stack`, `unstack`.

Namo — not currently planned. May be revisited when a concrete use case emerges. Namo's selection, projection, and composition operators cover many of the scenarios that motivate pivoting in other tools.

Sorting

Pandas — `df.sort_values('close')`.

Polars — `df.sort('close')`.

R/dplyr — `arrange(df, close)`.

xarray — `ds.sortby('close')`.

Julia/DataFrames.jl — `sort(df, :close)`.

Namo — `sort_by` via Enumerable. Will return a Namo as of 0.14.0.

Missing value handling

Pandas — `fillna`, `dropna`, `isna`.

Polars — `fill_null`, `drop_nulls`, `is_null`.

R/dplyr — `na.rm`, `drop_na`, `replace_na`.

xarray — `fillna`, `dropna`, `isnull`.

Julia/DataFrames.jl — `dropmissing`, `coalesce`, `ismissing`.

Namo — handles nils through Ruby. `nil` is a value like any other, and formulae must handle it explicitly (`v && v < 15`).

Vectorised operations

Pandas, **Polars**, **xarray**, **Julia** — operate on entire columns at once. Vectorised arithmetic runs at C/Fortran speed.

Namo — currently operates row-by-row in pure Ruby. This is the tradeoff that enables lazy formulae and arbitrary Ruby in procs. Columnar storage (planned for 3.x) and optional C acceleration will narrow the performance gap for selection-heavy and numeric workloads while preserving the row-level formula model.

Built-in I/O

Pandas — reads CSV, Excel, SQL, Parquet, JSON, HTML, and more.

Polars — reads CSV, Parquet, JSON, IPC.

R — reads CSV, Excel, databases.

xarray — reads NetCDF, GRIB, Zarr.

Julia/DataFrames.jl — reads CSV, Arrow, databases.

Namo — takes an array of hashes. The I/O is Ruby's job. Loaders planned for 1.2 as optional sugar.

Column-level operations

Pandas — `df['close'].mean()`, `df['close'].std()`, `df['close'].pct_change()`.

Polars — `df['close'].mean()`, expression-based column operations.

R/dplyr — `mean(df$close)`, `sd(df$close)`.

xarray — `ds['close'].mean()`, array-level operations.

Julia/DataFrames.jl — `mean(df.close)`, broadcasting.

Namo — `values[:dimension]` extracts a column as an array. No built-in column-level statistics. This is the domain of the companion `statistics.rb` gem.